

ECOM EXPERTS

Ecomofuscar

Manual de usuario, tecnico y de desarrollo

ecomofuscar 1.0.8 · 07/09/2026

Contenido

Coding Standards

Architecture

Hooks utilized

Modifying/Adding Algorithms

Introducción

Configuración Paso a Paso

Imágenes y lazy-load (a partir de v1.0.6)

Herramientas

Recomendaciones

Resumen

Algoritmos Disponibles

Funcionamiento del Modo OnClick

Targets y detección

Target product_thumbnail (v1.0.6)

Ofuscación de imágenes (v1.0.6)

Patrón de actualización (addnewfeatures)

Requisitos

Coding Standards

- **Standard:** PrestaShop 8 Coding Standards.
- **Compatibility:** PHP 7.4 - 8.2.
- **No Composer:** Dependencies are avoided to ensure stability across different hosting environments.

Architecture

- `classes/ObfuscationEngine.php` : Handles the core encoding/decoding logic.
- `classes/UrlRewriter.php` : Intercepts and rewrites HTML output and individual URLs via hooks.
- `classes/UrlMapper.php` : Manages the database persistence of obfuscated IDs.
- `views/js/obfuscator.js` : Handles frontend click interception and dynamic link rewriting for SEO protection.

Hooks utilized

- `actionDispatcher` : Resolves incoming obfuscated URLs before standard routing.
- `actionOutputHTMLBefore` : The primary hook for rewriting the DOM.
- `actionFacetedSearchOverrideUrl` : Specific integration for PrestaShop's `ps_facetedsearch` module.

Modifying/Adding Algorithms

To add a new algorithm: 1. Register it in `ecomofuscar.php :: $algorithms` . 2. Implement the logic in `ObfuscationEngine.php` . 3. Update `UrlRewriter.php` to handle any specific HTML transformations needed.

Guía de Usuario - Expert Link Obfuscator

Introducción

Este módulo protege su estrategia SEO ocultando enlaces que no desea que Google rastree (filtros, paginaciones, páginas de login, CMS legales, subpáginas de cuenta, etc.), así como las propias imágenes de producto si lo desea.

Configuración Paso a Paso

1. **Activar el módulo:** Asegúrese de que el interruptor principal esté en "Sí".
2. **Seleccionar Algoritmo:** - **JavaScript OnClick:** máxima protección (los bots no siguen `<span onclick>`). - **Short ID:** enlaces cortos tipo `/x/abcd123`, indescifrables sin acceso a la BD. - **Base64:** enlaces legibles sólo si se decodifica el payload.
3. **Contextos (targets) a ofuscar:** - **Facetas, paginación, orden:** filtros laterales, `?page=`, `orderby`. - **Buscador interno, newsletter, redes sociales.** - **CMS secundarios:** aviso legal, cookies, devoluciones, etc. Si no marca ninguno en la lista, se ofuscan todos automáticamente. - **Subpáginas de cuenta:** direcciones, historial, identidad, contraseña, order-slip, guest-tracking. - **Imágenes:** oculta las URLs de `/img/p/...` para dificultar el scraping de fotografías de producto. - **Miniaturas de producto (v1.0.6):** convierte los enlaces `<a>` del listado (categoría, búsqueda, home) en `<span onclick>`, para que Google no siga ese camino duplicado desde categorías a ficha. La foto de la miniatura se mantiene visible con su URL real. **Importante:** si activas este target, **desactiva "Imágenes"**, así te queda el patrón `<span onclick>` + foto real.

Imágenes y lazy-load (a partir de v1.0.6)

- La ofuscación de imágenes ya es **compatible con lazy-load** (tanto `loading="lazy"` nativo como librerías con `data-src` / `data-original`).
- Si venía de una versión anterior y las imágenes aparecían en blanco al activar el target "Imágenes", actualice a 1.0.6, entre al panel del módulo (se auto-aplican las mejoras) y **vacíe la caché de PrestaShop** para que los templates se regeneren sin los placeholders antiguos.
- Ya no es necesario tocar nada en el tema: el módulo reescribe tanto `src` como los atributos `data-src` más comunes.

Herramientas

- **Vaciar caché:** limpia la tabla de mapeos URL→payload (útil tras cambiar muchas URLs).
- **Regenerar claves:** renueva los secretos internos si sospecha que alguien está decodificando sus URLs ofuscadas.

Recomendaciones

- Tras activar/desactivar targets, limpie siempre la caché de Smarty de PrestaShop.
- Deje visible al menos "Quiénes somos" o "Contacto" si quiere que Google las indexe: para ello, marque el resto de CMS en la lista de "Páginas CMS secundarias".

Guía Técnica - Expert Link Obfuscator v1.0.6

Resumen

El módulo `ecomofuscar` permite ocultar enlaces a los motores de búsqueda mediante diversas técnicas de ofuscación, mejorando el SEO al evitar el rastreo de facetas, paginaciones, páginas CMS secundarias, subpáginas de cuenta y otros enlaces internos sin valor de indexación.

Algoritmos Disponibles

1. **Sin ofuscar (`none`)**: Mantiene los enlaces originales.
2. **Base64 (`base64`)**: Codifica la URL en base64 y la sirve a través de un controlador interno (`/x/payload`).
3. **Short ID (`shortid`)**: Mapea la URL a un identificador corto persistido en BD (máxima seguridad: no se decodifica sin acceso a la tabla de mapeos).
4. **JS OnClick (`onclick`)**: Transforma los enlaces `<a>` en `<span>` con un `onclick` que decodifica la URL en el cliente con `atob` y redirecciona.

Funcionamiento del Modo OnClick

Snippet generado por cada `<a>` detectado:

```
<span
onclick="window.location.href=unescape(decodeURIComponent(window.atob('base64_url')))">Texto</span>
```

Targets y detección

- La detección de contexto está en `EcomOfuscarUrlRewriter::detectContext($url, $attrs, $tagName)`.
- Para friendly URLs (`/content/...`, subpáginas de cuenta localizadas) se leen los slugs reales desde `ps_meta_lang` y `ps_cms_lang` (caché estática por request).
- `cms_secondary`: si la lista `CMS_SECONDARY_IDS` está vacía se ofuscan **todas** las CMS detectadas; si está llena, sólo las marcadas.

Target `product_thumbnail` (v1.0.6)

Convierte los `<a>` que envuelven la miniatura de producto en listados (categoría, búsqueda, home) en `<span onclick>` siguiendo el algoritmo activo (onclick con base64 decode, o useJsFrontend con `data-go`). La imagen interior NO se toca: conserva su `src` real y carga sin pasar por el dispatcher.

Detección (en `EcomOfuscarUrlRewriter::detectContext` para `$tagName === 'a'`), por orden (primera que casa, gana):

- Por clase conocida:** `product-thumbnail`, `product-miniature-link`, `product-miniature`, `js-product-miniature`, `home-product-thumbnail`, `product_img_link`, `product_name_link`, `product-name-link`, `product_img`, `product-img-link`, `product-image-link`, `product-image`, `product-link`, `product-cover`, `js-qv-product-cover`, `tm_gallery_item_box`, `tm_product_link`, `prod_thumb_link`, `thumbnail-product`, `item-product`, `card-product`, `item-img`, `item-image`.
- Heurístico potente por `<img>` hijo:** href coincide con patrón de URL de ficha PS `#/\d+(?:-\d+)?-[a-z0-9][a-z0-9]*\.html#` (cubre `/3740-slug.html` y combinaciones `/3740-3979-slug.html`) **AND** el `<a>` envuelve un `<img>` (se chequea `$innerHtml` recibido en la callback). Esta regla es la más robusta: cubre cualquier tema custom que no ponga clase al enlace de la miniatura.
- Fallback por clase genérica:** href coincide con patrón de ficha PS **AND** la clase contiene `thumbnail|miniature|gallery|product-|product_|card-img|item-img|image-block`. Cubre el `<a>` del título (sin `<img>` hijo) en listados con clases propias.

En 2 y 3, siempre se excluyen anchors cuya clase contenga `breadcrumb|ajax_add_to_cart|add-to-cart|add_to_cart`.

La misma lógica está replicada en `views/js/obfuscator.js::detectContext(url, el)` para que contenido cargado vía AJAX (infinite scroll, paginación AJAX, quickview) reciba el mismo trato que el HTML renderizado por el hook server-side.

Combinar con: - **Algoritmo:** `onclick` (más seguro, base64 en el snippet) o `base64` (`src /x/{payload}`). - **DESACTIVAR target `images`** si lo que quieres es el patrón "`<span onclick> + <img>` real". Si dejas `images` activo, el `src` del `<img>` se reescribirá al dispatcher y perderás la foto directa.

Ofuscación de imágenes (v1.0.6)

- Antes ($\leq 1.0.5$) se usaba un placeholder base64 1x1 (`tinygif`) en `src` + `data-go` con la URL real, y el JS de frontend restauraba `src` al cargar el DOM.
- Problema: rompía el lazy-load (ya fuese atributo `loading="lazy"` nativo o librerías con `data-src` / `data-original`), dejando imágenes en blanco o mostrando placeholders en vez del archivo real.
- Fix v1.0.6: el `<img>` se reescribe apuntando `src` directamente al dispatcher `/x/{payload}`. El dispatcher hace `302` al archivo real, así el navegador (y cualquier librería de lazy-load) lo trata como una URL normal.
- Adicionalmente se reescriben los atributos lazy más comunes y los de zoom/lightbox: `data-src`, `data-original`, `data-lazy-src`, `data-lazy`, `data-echo`, `data-full-size-image-url`, `data-zoom-image`, `data-large-image`. De este modo la URL real no aparece en el HTML ni siquiera en los atributos de lazy-load o zoom.
- `srcset` se elimina del `<img>`: ofuscar cada candidate-URL (`url 1x`, `url 2x`, `url 800w`) multiplicaría URLs expuestas; dejar sólo `src` apuntando al dispatcher es más limpio y el navegador lo usa como fallback.
- `processImages()` en `views/js/obfuscator.js` se conserva por compatibilidad con HTML cacheado previo a la actualización, pero ya no es necesario limpiar Smarty para que la nueva ofuscación funcione sobre páginas nuevas.

Patrón de actualización (`addnewfeatures`)

Los defaults y registros de hooks/tablas nuevos van en el método `addnewfeatures()` de la clase principal. Este método es **idempotente** y se llama tanto desde `install()` como desde `getContent()` , de modo que las tiendas ya actualizadas (sin reinstalar) adquieren los nuevos ajustes al abrir el panel del módulo.

Requisitos

- PrestaShop 1.7 o 8.x
- PHP 7.4 o superior